WRITE-UP FINAL ARKAVIDIA 10.0

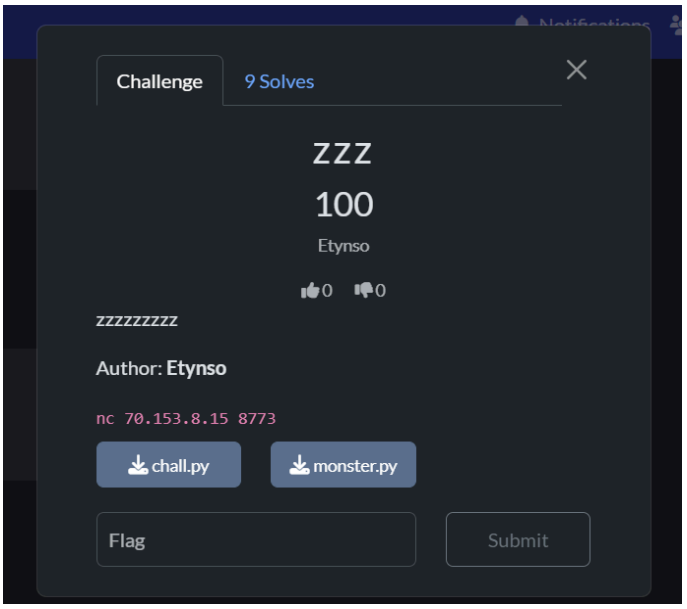
(SMEAS - 眼鏡をかけたフィネシットショートヘアのおてんば娘が欲しい)

ANGGOTA : 1. HENALVARO FARREL BAGAS QUSUMA (KETUA)
2. PARAMA WIDYADHANA PUTRA ARIF
3. SURYA EGA ATHA WIRAWAN

ASAL : SMK NEGERI 1 SURABAYA

CRYPTOGRAPHY

1. ZZZ



Pada challenge ini, peserta diberikan implementasi RSA standar melalui file chall.py serta fungsi hashing kustom pada file monster.py. Tujuan utama challenge adalah merekonstruksi sebuah secret 512-bit yang dienkripsi menggunakan RSA, kemudian mengirimkannya kembali ke server untuk memperoleh flag.

Challenge ini menarik karena bukan menyerang RSA secara langsung, melainkan mengeksploitasi kelemahan desain oracle yang disediakan.

Server menghasilkan sebuah secret acak berukuran 512-bit:

secret = getRandomRange(0, n)

Kemudian server mengirimkan parameter publik RSA:

- n
- e
- enc = secret^e mod n

Ciphertext dikirim dalam format heksadesimal:

Encrypted: 0x{pow(secret, e, n):x}

Tugas kita adalah merekonstruksi nilai secret tersebut.

Server menyediakan oracle interaktif dengan mekanisme sebagai berikut:

1. Client mengirim ciphertext c.
2. Server mendekripsi ciphertext menjadi val.
3. Server mengambil window 8-bit berturut-turut dari val, dimulai dari indeks k.
4. Client dapat mengirim permutasi dari 8 bit tersebut.
5. Bit hasil permutasi dimasukkan ke fungsi hashing kustom chomp(*bits).

6. Output chomp berupa satu bit (0 atau 1).

Batas total interaksi adalah 15 × 512.

Fungsi yang menjadi pusat kerentanan adalah chomp:

def chomp(x, y, z, w, u, v, s, t):
*result = hashlib.blake2s(str((u * 3081 + (1 - u) * 3276 + ...)).encode()).digest()[11] & 1*
result ^= ...

Parameter kelima adalah u.

Melalui analisis matematis dan brute-force terhadap seluruh 128 kombinasi parameter lainnya, diperoleh sifat penting berikut:

- Jika u = 0, output chomp dapat bernilai 0 atau 1 dengan probabilitas sekitar 48% untuk menghasilkan 1.
- Jika u = 1, output chomp selalu 0.

Implikasi:

Jika output chomp bernilai 1, maka dapat dipastikan bahwa u = 0.

Hal ini menghasilkan oracle satu arah (one-sided oracle), di mana:

- Output 1 memberikan kepastian informasi.
- Output 0 tidak memberikan kepastian.

Karena kita dapat menentukan permutasi 8 bit sebelum masuk ke chomp, kita bisa memindahkan bit target dari window 8-bit ke posisi parameter u.

Contoh permutasi:

1 2 3 4 0 5 6 7

Dengan permutasi ini, bit pertama dari window akan menjadi parameter u.

Langkah untuk setiap indeks bit k:

1. Kirim ciphertext asli enc.
2. Kirim permutasi yang menempatkan bit target pada posisi u.
3. Jika output adalah 1, maka bit ke-k pasti bernilai 0.

Tidak diperlukan manipulasi ciphertext seperti pada RSA LSB Oracle Attack klasik. Kita hanya memanfaatkan dekripsi langsung dari ciphertext secret.

Karena probabilitas output 1 saat u = 0 sekitar 48%, maka:

- Jika suatu bit bernilai 0, dalam beberapa percobaan hampir pasti akan menghasilkan output 1.
- Jika suatu bit bernilai 1, tidak akan pernah menghasilkan output 1.

Strategi yang digunakan:

- Lakukan beberapa pass penuh terhadap seluruh 512 bit.
- Jika suatu bit pernah menghasilkan output 1, tandai sebagai 0.
- Jika setelah beberapa pass tidak pernah menghasilkan 1, tandai sebagai 1.

Dalam praktik, sekitar 6–10 pass sudah cukup untuk memastikan klasifikasi seluruh bit dengan probabilitas kegagalan yang sangat kecil.

Melakukan interaksi satu per satu menyebabkan latency jaringan tinggi.

Untuk mengatasi hal ini, digunakan teknik pipelining:

- Kirim seluruh 512 query dalam satu batch.
- Setelah itu, baca seluruh respons secara berurutan.

Pendekatan ini secara signifikan mengurangi round-trip delay dan mempercepat proses eksploitasi.

Setelah semua pass selesai:

- Bit yang pernah menghasilkan output 1 ditandai sebagai 0.
- Bit yang tidak pernah menghasilkan output 1 dianggap sebagai 1.

Secret kemudian direkonstruksi dengan menyusun ulang seluruh bit sesuai indeksnya.

Setelah secret berhasil direkonstruksi, nilai tersebut dikirim ke server untuk memperoleh flag.

Secret berhasil direkonstruksi secara penuh.

script yang digunakan:

```
import sys

import hashlib

from pwn import *

import random



# Fix path to import monster

sys.path.append('.')

from monster import chomp



def get_perms():

    # Use deterministic random or predefined set for consistency

    random.seed(42)

    perms = []

    for _ in range(15):

        p = list(range(8))

        random.shuffle(p)
```

```
        perms.append(p)

    return perms


def precompute_sigs(perms):

    sigs = {}

    for i in range(256):

        # i is the window value 0..255

        # Bit j is (i >> j) & 1

        window_bits = [(i >> j) & 1 for j in range(8)]

        sig = tuple(chomp(*(window_bits[p] for p in perm)) for perm in perms)

        sigs.setdefault(sig, []).append(i)

    return sigs


def solve_bits(candidates, NBITS=512):

    # candidates[k] = list of possible values for window starting at index k

    secret_bits = [None] * NBITS


    def backtrack(k):

        if k == NBITS:

            return True


        # Candidates for window k must match existing bits

        for val in candidates[k]:

            match = True

            # Check consistency of val bits [0..7] with
secret_bits[(k+l)%NBITS]

            for l in range(8):

                idx = (k + l) % NBITS

                if secret_bits[idx] is not None and ((val >> l) & 1) !=
secret_bits[idx]:
```

```
                    match = False
                    break

            if match:
                # Save old values to restore
                added = []
                for l in range(8):
                    idx = (k + l) % NBITS
                    if secret_bits[idx] is None:
                        secret_bits[idx] = (val >> l) & 1
                        added.append(idx)

                if backtrack(k + 1):
                    return True

                # Restore
                for idx in added:
                    secret_bits[idx] = None
        return False

    if backtrack(0):
        # Convert bits to int
        secret = 0
        for i in range(NBITS):
            if secret_bits[i]:
                secret |= (1 << i)
        return secret
    return None
```

```python
def main():
    HOST = '70.153.8.15'
    PORT = 8773

    perms = get_perms()
    sigs = precompute_sigs(perms)

    io = remote(HOST, PORT)

    line = io.recvline().decode()
    print(line)
    # n, e = (..., 65537)
    parts = line.split('=')[1].strip()[1:-1].split(',')
    n = int(parts[0])
    e = int(parts[1])

    line = io.recvline().decode()
    print(line)
    # Encrypted: 0x...
    enc_secret = int(line.split(': ')[1], 16)

    NBITS = 512
    results = [[None] * NBITS for _ in range(15)]

    # Progress Bar-ish
    for i in range(15):
        print(f"Starting pass {i}...")
        p_str = " ".join(map(str, perms[i]))
        for k in range(NBITS):
```

```
        io.sendlineafter(b"> ", b"1")
        io.sendlineafter(b"enc: ", hex(enc_secret).encode())
        io.sendlineafter(b"Permutation: ", p_str.encode())
        res = io.recvline().decode()
        # dec: 0x...
        val = int(res.split(': ')[1], 16)
        results[i][k] = val

# Analyze
candidates = []
for k in range(NBITS):
    obs = tuple(results[i][k] for i in range(15))
    if obs not in sigs:
        print(f"Error: Signature {obs} not found for k={k}!")
        # This shouldn't happen unless noise or wrong model
        candidates.append(list(range(256)))
    else:
        candidates.append(sigs[obs])

print("Recovering secret...")
secret = solve_bits(candidates)
if secret is not None:
    print(f"Found secret: 0x{secret:x}")
    io.sendlineafter(b"> ", b"2")
    io.sendlineafter(b"secret: ", hex(secret).encode())
    print(io.recvall().decode())
else:
    print("Failed to recover secret bits.")
```

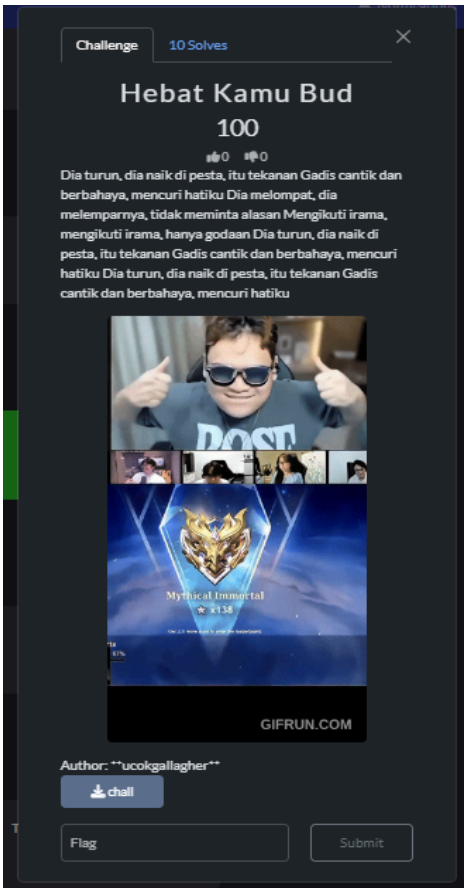
```
if __name__ == "__main__":

    main()
```

Flag yang diperoleh:

ARKAV{92f907a1c476abdb1cca3da49d79488681d32deee97c6687aed1526854fa6ca6}

REVERSE ENGINEERING

1. HEBAT KAMU BUD



Langkah pertama dalam analisis binary adalah mengidentifikasi karakteristik file yang diberikan. Gunakan perintah berikut:

file chall

Hasil:

ELF 64-bit LSB executable, x86-64, version 1 (GNU/Linux), statically linked, not stripped

Interpretasi:

- File merupakan executable Linux 64-bit.
- Bersifat statically linked, sehingga seluruh library sudah tertanam dalam binary.
- Status not stripped berarti simbol dan nama fungsi asli masih tersedia, sehingga proses reverse engineering menjadi jauh lebih mudah.

Binary kemudian dibuka menggunakan Ghidra dan dilakukan auto-analysis untuk mendapatkan struktur fungsi dan alur program.

Pada Symbol Tree ditemukan fungsi main yang memiliki alur sebagai berikut:

1. Program meminta input flag melalui printf("flag? ").
2. Input diproses oleh fungsi flag_to_digits.
3. Dibuat array digits_b berisi angka 9 sampai 1.
4. Data diproses oleh fungsi perform_computation.
5. Hasil diverifikasi oleh verify_result.

Struktur ini menunjukkan adanya proses transformasi numerik terhadap input sebelum diverifikasi.

Pada fungsi ini ditemukan pengecekan panjang input:

```
if ((int)uVar1 != 0x48) { return 0; }
```

Nilai 0x48 dalam heksadesimal sama dengan 72 dalam desimal.

Artinya, flag wajib memiliki panjang tepat 72 karakter. Jika tidak, program langsung menghentikan eksekusi dan menampilkan pesan gagal.

Fungsi ini mengimplementasikan Number Theoretic Transform (NTT), yang pada dasarnya merupakan perkalian polinomial dalam domain modular.

Secara matematis:

$$C(x) = A(x) \cdot B(x) \bmod 2013265921$$

Dengan:

- A(x) berasal dari representasi numerik flag.
- B(x) adalah array tetap [9, 8, 7, 6, 5, 4, 3, 2, 1].

Output dari perkalian polinomial inilah yang kemudian diverifikasi pada tahap berikutnya.

Permasalahan muncul pada fungsi verify_result.

Program tidak membandingkan hasil dengan data statis yang tersimpan dalam binary, melainkan dengan nilai yang dibentuk saat runtime. Hal ini menyulitkan pendekatan statis murni.

Solusi yang digunakan adalah memanfaatkan GDB untuk mengekstrak nilai yang diharapkan program.

Dari Ghidra ditemukan instruksi perbandingan (CMP) pada alamat:

- 0x00403c56

Buat file bernama solve.gdb dengan isi berikut:

```
- break *0x00403c56
- commands
- silent
- printf "%d, ", $eax
- set {int}($r14 + $r13 * 4) = $eax
- continue
- end
```

Penjelasan:

- Program berhenti setiap kali mencapai instruksi CMP.
- Register EAX berisi nilai yang diharapkan.
- Nilai tersebut dicetak.
- Memory input dipaksa agar sesuai dengan nilai yang diharapkan sehingga loop dapat terus berjalan.

Langkah eksekusi:

- chmod +x chall
- gdb -x solve.gdb ./chall

Ketik run, lalu masukkan dummy flag sepanjang 72 karakter, misalnya:

ARKAV{0123456789012345678901234567890123456789012345678901234567890123456789012345678901234}

GDB akan mencetak 227 angka. Deretan angka inilah yang menjadi target hasil perkalian polinomial.

Setelah memperoleh array hasil target, langkah berikutnya adalah membalik proses perkalian polinomial menggunakan teknik pembagian polinomial.

Contoh implementasi dalam Python:

```
target_c = [45, 58, 60, 133, 152, 131, 182, 207, 172, 182, 219, 178, 192, 209,
170, 176, 221, 182, 206, 174, 149, 195, 157, 134, 119, 134, 108, 145, 119, 103,
123, 183, 153, 178, 154, 134, 122, 142, 112, 155, 204, 172, 149, 167, 136, 142,
110, 92, 109, 165, 140, 160, 131, 116, 118, 94, 79, 82, 60, 56, 42, 42, 41, 76,
64, 61, 103, 97, 90, 119, 179, 150, 122, 108, 93, 87, 112, 88, 65, 56, 55, 62,
94, 77, 124, 103, 96, 88, 115, 93, 126, 110, 98, 121, 182, 153, 125, 107, 93,
132, 116, 100, 111, 96, 89, 135, 120, 105, 135, 121, 107, 129, 186, 154, 141,
160, 130, 182, 186, 151, 117, 88, 77, 101, 158, 130, 165, 139, 126, 148, 120,
101, 91, 111, 90, 69, 64, 58, 88, 154, 130, 170, 223, 191, 240, 235, 191, 148,
110, 90, 105, 158, 130, 174, 183, 156, 192, 203, 164, 198, 193, 157, 121, 102,
86, 60, 41, 36, 102, 149, 130, 156, 213, 181, 168, 180, 142, 150, 180, 147,
132, 165, 137, 136, 179, 147, 160, 200, 167, 188, 160, 138, 151, 171, 139, 188,
152, 133, 131, 123, 105, 132, 176, 146, 161, 189, 157, 171, 201, 163, 144, 183,
150, 162, 183, 150, 135, 132, 106, 80, 56, 40, 24, 13, 8, 3, 0]

b = [9, 8, 7, 6, 5, 4, 3, 2, 1]

a = []



for i in range(len(target_c) - len(b) + 1):

val = target_c[i] // b[0]

a.append(val)

for j in range(len(b)):

target_c[i+j] -= val * b[j]
```

```
flag = ""
for i in range(0, len(a), 3):
char_code = a[i+2]*100 + a[i+1]*10 + a[i]
if 32 <= char_code <= 126:
flag += chr(char_code)

print("Flag Found:", flag[::-1])
```

Metode ini membalik proses konvolusi polinomial untuk memperoleh kembali koefisien asli yang merepresentasikan karakter flag.

Flag yang diperoleh:

ARKAV{mAtUSHK4_Odn090_d1a_n4ik_d14_turun_t3k4n4n_tinggi_h3b4t_k4mu_BUD1}

TERIMA KASIH